

编译原理 Ch3 ——语法分析考试复习总结

中南大学陈志刚教授课件整理

2026 年 5 月

目录

1 语法分析简介	1
2 上下文无关文法	1
3 语法树与二义性	3
4 自顶向下分析	3
5 自底向上分析	8
6 语法分析算法总结与比较	12
7 核心考点速记	13

1 语法分析简介

语法分析器在编译器中的位置

语法分析器 (Syntax Analyzer / Parser) 位于编译器的前端, 处于词法分析器之后、语义分析器之前。词法分析器将源代码转换为记号流 (Token Stream), 语法分析器接收记号流并输出语法树 (Syntax Tree), 语义分析器再基于语法树进行后续处理。[p.5-6]

语法分析器的任务

给定语言语法规则和记号流, 语法分析器的核心任务是回答: 该记号流是否可以由给定文法产生? 如果可以, 则构造语法树; 如果不可以, 则报告语法错误。[p.6, p.33]

形式上: 给定文法 G 和记号流构成的句子 α , 问是否存在从开始符号 S 到 α 的推导。[p.32]

为什么不能用正则表达式?

正则表达式无法描述嵌套/递归结构。例如, 对于可能带括号的算式, 正则表达式无法记住前面是否出现过左括号。因此需要表达能力更强的工具——上下文无关文法 (CFG)。[p.13-14]

语法分析学习路线图

- 数学理论：上下文无关文法（CFG）
- 自顶向下分析：递归下降分析算法、LL 分析算法
- 自底向上分析：LR 分析算法、算符优先分析算法

[p.9]

2 上下文无关文法

乔姆斯基文法体系

Noam Chomsky 将文法分为四个层次：[p.14]

1. 0-型文法（无限制文法）： $\alpha \rightarrow \beta$ ， α 至少含一个非终结符
2. 1-型文法（上下文有关文法）： $\gamma_1 A \gamma_2 \rightarrow \gamma_1 \delta \gamma_2$ ，要求 $|\alpha| \leq |\beta|$
3. 2-型文法（上下文无关文法）： $A \rightarrow \delta$ ， $A \in V_N$ ， $\delta \in (V_N \cup V_T)^*$
4. 3-型文法（正规文法）： $A \rightarrow \alpha B$ 或 $A \rightarrow \alpha$ （右线性）

四个文法类逐渐增加限制，包含关系为：3-型 $\subset$ 2-型 $\subset$ 1-型 $\subset$ 0-型。现今大多数高级程序设计语言使用上下文无关文法描述语法已足够。[p.20]

上下文无关文法的形式化定义

一个上下文无关文法 G 是一个四元组 $G = (T, N, P, S)$ ：[p.26]

- T ：终结符（Terminal）集合
- N ：非终结符（Non-terminal）集合
- P ：产生式规则集合，每条规则形式为 $X \rightarrow \beta_1 \beta_2 \dots \beta_n$ ，其中 $X \in N$ ， $\beta_i \in (T \cup N)$
- S ：唯一的开始符号， $S \in N$

也可以记为 $G = (V_N, V_T, S, P)$ ，其中 $V_N \cap V_T = \emptyset$ ， $S \in V_N$ 。产生式一般形式： $A \rightarrow \alpha \mid \beta$ 。[p.15]

2-型文法的扩充

允许产生式右部为空串： $A \rightarrow \varepsilon$ 。即 $\delta \in (V_N \cup V_T)^*$ ，而非强制 $+$ 。[p.19]

BNF 范式

BNF（Backus-Naur Form）是上下文无关文法的一种实现表示方法。非终结符用尖括号表示，如 $\langle E \rangle$ ，终结符直接写出，如 id 。产生式用 $::=$ 表示。[p.29]

推导

给定文法 G ，从开始符号 S 开始，用产生式右部替换左侧的非终结符，反复进行直到不出现非终结符为止，最终的串称为句子。[p.30]

最左推导与最右推导

- 最左推导 (Leftmost Derivation): 每次总是选择最左侧的非终结符进行替换。[p.31]
- 最右推导 (Rightmost Derivation): 每次总是选择最右侧的非终结符进行替换。最右推导是自底向上 LR 分析的基础。[p.31]

句型、句子、语言

- 句型: 设 S 是文法 G 的开始符号，若 $S \xRightarrow{*} u$ ，则 u 为句型 (u 可含非终结符)。[p.42]
- 句子: 若 $S \xRightarrow{*} u$ 且 $u \in V_T^*$ ，则 u 为句子。[p.42]
- 语言: $L(G) = \{u \mid S \xRightarrow{*} u \text{ 且 } u \in V_T^*\}$ ，即文法所有句子的集合。[p.42]

直接推导

若存在规则 $A \rightarrow \beta$ ，且 $\gamma, \delta \in (V_N \cup V_T)^*$ ，则有: $\gamma A \delta \Rightarrow \gamma \beta \delta$ ，称为直接推导。[p.44]

文法等价

若 $L(G_1) = L(G_2)$ ，则称文法 G_1 和 G_2 等价。[p.47]

3 语法树与二义性

推导与语法树的对应

推导可以表达成树状结构——语法树。不论推导所用的顺序 (最左、最右或其他)，都可以表示成语法树。[p.39-40]

- 树中的每个内部节点代表非终结符
- 每个叶子节点代表终结符
- 每一步推导代表如何从父节点生成它的直接孩子节点

语法树的含义取决于树的后序遍历。[p.41]

二义性文法

给定文法 G ，如果存在句子 s 有两棵不同的语法树 (即有不同的推导)，则称 G 是二义性文法。[p.50]

从编译器角度看，二义性文法的同一程序会有不同含义，导致运行结果不唯一。

解决方案: 文法的重写 [p.50-52]

例如，将二义性的表达式文法:

$E \rightarrow \text{num} \mid \text{id} \mid E+E \mid E * E$

重写为无二义性文法，通过引入优先级层次：

$E \rightarrow E+T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow \text{num} \mid \text{id}$

这样 $*$ 的优先级高于 $+$ ，且 $+$ 为左结合。[p.51-52]

4 自顶向下分析

自顶向下分析的算法思想

从文法的开始符号出发，尝试选择产生式推导出某个句子 t ，比较 t 和要分析的句子 s 。若 $t = s$ 则接受，否则回溯尝试其他产生式。[p.56]

基本算法需要回溯（Backtracking），效率低。实际编译器需要线性时间的算法，于是引出了递归下降分析算法和 LL(1) 分析算法。[p.60]

4.1 递归下降分析

基本思想

递归下降分析也称为预测分析（Predictive Parsing）。[p.63]

- 每个非终结符构造一个分析函数
- 对产生式右部的每个非终结符调用其分析函数
- ”吃掉”产生式右部的每个终结符
- ”|” 转换为”else-if” 分支，用前看符号指导分支选择

[p.66]

递归下降分析的特点：

- 分析高效（线性时间）
- 容易手工编码实现
- 错误定位和诊断信息准确
- 被 GCC 4.0、LLVM 等众多编译器采用

[p.63]

算法步骤

对每个非终结符 X 及其产生式 $X \rightarrow \beta_{11} \dots \beta_{1i} \mid \beta_{21} \dots \beta_{2j} \mid \dots$:

1. 构造一个分析函数
2. 根据当前的前看符号选择匹配的候选式
3. 对候选式右部的终结符进行匹配（”吃掉”）
4. 对候选式右部的非终结符递归调用其分析函数

[p.64-67]

预处理：提取左公因子

当产生式有公共前缀时，前看符号无法指导分支选择。[p.68]

$A \rightarrow aB1 \mid aB2 \mid aB3 \mid \dots$

提取左公因子后：

$A \rightarrow aA'$

$A' \rightarrow B1 \mid B2 \mid B3 \mid \dots$

预处理：消除左递归

左递归会使递归下降分析器陷入无限递归。[p.69]

直接左递归的消除：[p.72] 若 $P \rightarrow P\alpha \mid \beta$ ($\alpha \neq \varepsilon$, β 不以 P 开头), 改写为:

$P \rightarrow \backslash\text{beta } P'$

$P' \rightarrow \backslash\text{alpha } P' \mid \backslash\text{varepsilonpsilon}$

一般情况：若 $P \rightarrow P\alpha_1 \mid P\alpha_2 \mid \dots \mid P\alpha_m \mid \beta_1 \mid \dots \mid \beta_n$, 改写为：[p.73]

$P \rightarrow \backslash\text{beta_1 } P' \mid \backslash\text{beta_2 } P' \mid \dots \mid \backslash\text{beta_n } P'$

$P' \rightarrow \backslash\text{alpha_1 } P' \mid \backslash\text{alpha_2 } P' \mid \dots \mid \backslash\text{alpha_m } P' \mid \backslash\text{varepsilonpsilon}$

示例：

$E \rightarrow E+T \mid T \Rightarrow E \rightarrow TE'$

$E' \rightarrow +TE' \mid \backslash\text{varepsilonpsilon}$

$T \rightarrow T*F \mid F \Rightarrow T \rightarrow FT'$

$T' \rightarrow *FT' \mid \backslash\text{varepsilonpsilon}$

[p.70]

完全消除左递归（包括间接左递归）：[p.74-76] 对不含回路和 ε 产生式的文法：

1. 将非终结符按任意顺序排列为 $P_1, P_2, \dots, P_n$
2. 对 $i = 1$ 到 n , 对 $j = 1$ 到 $i - 1$, 把形如 $P_i \rightarrow P_j\gamma$ 的规则改写成 $P_i \rightarrow \delta_1\gamma \mid \dots \mid \delta_k\gamma$
3. 消除关于 P_i 的直接左递归
4. 化简，去掉无用非终结符产生式

4.2 LL(1) 分析算法

LL(1) 的含义

LL(1): 从左 (L) 向右读入程序, 最左 (L) 推导, 采用一个 (1) 前看符号。[p.85]

特点: 表驱动的分析算法, 分析高效 (线性时间), 错误定位准确, 适用于自动生成语法分析器 (如 ANTLR)。[p.85]

表驱动的 LL 分析器架构

分析器由分析栈、分析表、驱动代码组成。分析表告诉分析器: 当栈顶是某非终结符且输入是某终结符时, 应使用第几条产生式替换 (展开)。[p.86-88]

FIRST(N) 集

从非终结符 N 开始推导得出的句子开头的所有可能终结符的集合。[p.92]

近似公式: [p.92]

- 对 $N \rightarrow a \dots$: $\text{FIRST}(N) \cup = \{a\}$
- 对 $N \rightarrow M \dots$: $\text{FIRST}(N) \cup = \text{FIRST}(M)$

完整公式 (考虑 NULLABLE): [p.100]

- 对 $X \rightarrow a$: $\text{FIRST}(X) \cup = \{a\}$
- 对 $X \rightarrow Y_1 Y_2 \dots Y_n$:
 - $\text{FIRST}(X) \cup = \text{FIRST}(Y_1)$
 - 若 $Y_1 \in \text{NULLABLE}$, $\text{FIRST}(X) \cup = \text{FIRST}(Y_2)$
 - 若 $Y_1, Y_2 \in \text{NULLABLE}$, $\text{FIRST}(X) \cup = \text{FIRST}(Y_3)$
 - 以此类推

NULLABLE 集合

非终结符 X 属于 NULLABLE 当且仅当: [p.97]

- 基本情况: 存在 $X \rightarrow \varepsilon$
- 归纳情况: $X \rightarrow Y_1 \dots Y_n$, 且 $Y_1, \dots, Y_n$ 都属于 NULLABLE

FIRST_S 产生式集

一个产生式的 FIRST_S 集: $\text{FIRST_S}(\beta_1 \dots \beta_n)$ 定义为从该产生式右部开始推导得出的句子开头的所有可能终结符集合。[p.90]

计算方法 (完整版): [p.107]

- 若 $\beta_1 = a$ (终结符), 则 $\text{FIRST_S} \cup = \{a\}$
- 若 $\beta_1 = M_1$ (非终结符), 则:

- $\text{FIRST_S} \cup = \text{FIRST}(M_1)$
- 若 M_1 非 NULLABLE, 停止
- 若 M_1 NULLABLE, 继续 $\text{FIRST}(M_2)$, 以此类推
- 若所有 β_i 都 NULLABLE, 则 $\text{FIRST_S} \cup = \text{FOLLOW}(N)$, 其中 N 是该产生式的左部

FOLLOW 集

$\text{FOLLOW}(N)$ = 紧跟在非终结符 N 后面的终结符号的集合。[p.104]

不点算法: [p.105] 对每条产生式 $N \rightarrow \beta_1 \dots \beta_{n-2} \beta_{n-1} \beta_n$:

- 令 $\text{temp} = \text{FOLLOW}(N)$
- 从右向左扫描 β_i (i 从 n 到 1):
 - 若 $\beta_i = a$ (终结符), $\text{temp} = \{a\}$
 - 若 $\beta_i = M$ (非终结符):
 - * $\text{FOLLOW}(M) \cup = \text{temp}$
 - * 若 M 非 NULLABLE, $\text{temp} = \text{FIRST}(M)$
 - * 若 M NULLABLE, $\text{temp} = \text{FIRST}(M) \cup \text{temp}$

FIRST 和 FOLLOW 的不动点算法

两者均采用不动点迭代算法: 从空集开始, 反复应用产生式规则更新集合, 直到所有集合不再变化 (收敛到不动点)。[p.93, p.101-102, p.105-106]

构造 LL(1) 分析表

对每条产生式 $A \rightarrow \beta$:

- 对 $\text{FIRST_S}(\beta)$ 中的每个终结符 a : $\text{table}[A][a] = \text{该产生式编号}$
- 若 β 可以推导出 ε (即 NULLABLE), 对 $\text{FOLLOW}(A)$ 中的每个终结符 b : $\text{table}[A][b] = \text{该产生式编号}$

[p.109]

LL(1) 驱动的伪代码

```
stack.push($); stack.push(S);
while (栈非空) {
    X = stack.top();
    a = tokens[i];
    if (X == a) { pop(); i++; }
    else if (X是终结符 && X != a) ERROR;
    else if (table[X][a] != empty)
        pop(); 压入产生式右部(逆序);
```

```

    else ERROR;
}

```

[p.110]

4.3 LL(1) 分析冲突处理

LL(1) 文法的判定条件

文法 G 为 LL(1) 文法当且仅当 G 中的任意两条产生式 $A \rightarrow \alpha \mid \beta$ 满足: [p.113]

1. 当 α 与 β 都不能推导出 ε , 则 $\text{FIRST}(\alpha)$ 与 $\text{FIRST}(\beta)$ 不相交
2. 当 α 或 β 能推导出 ε :
 - 若 α 能推导出 ε , 则 $\text{FIRST}(\beta)$ 与 $\text{FOLLOW}(A)$ 不相交
 - 若 β 能推导出 ε , 则 $\text{FIRST}(\alpha)$ 与 $\text{FOLLOW}(A)$ 不相交

冲突消除策略

策略 1——FIRST 相交: 提取左公因子。[p.116] 将 $A \rightarrow \gamma\alpha_1 \mid \gamma\beta_2$ 改写为 $A \rightarrow \gamma A', A' \rightarrow \alpha_1 \mid \beta_2$ 。

策略 2——都含 ε : 删除某一产生式中的 ε 。[p.117]

策略 3—— ε 导致 FIRST 与 FOLLOW 相交: 展开产生式, 消除左递归或提取左公因子。[p.118-119]

- Case 1: 提取左公因子 (展开 ε 候选式)
- Case 2: 消除左递归
- Case 3: 某些情况下可能失败 (FIRST 与 FOLLOW 永远相交) [p.120]

无回溯文法的不可判定性

并非所有的 CFG 都有对应的无回溯文法。对于任意的 CFG, 其是否存在无回溯文法是不可判定的。[p.121]

5 自底向上分析

自底向上分析的基本思想

从待分析的句子出发, 尝试自底向上地规约成文法的产生式。自底向上分析中最重要/最广泛应用的是 LR 分析算法 (移进-归约算法), 由 Knuth 于 1965 年提出。[p.128]

LR 分析算法特点:

- 算法运行高效
- 广泛应用于语法分析器的自动生成器中: YACC, bison, CUP, C#yacc 等

自底向上推导的过程是最右推导的逆过程。[p.129]

点记号与 LR(0) 项目

为了方便标记语法分析器已经读入了多少输入，引入点记号“ $\bullet$ ”。一个产生式加入一个圆点，成为一个 LR(0) 项目 (Item)。[p.130]

例如： $E \rightarrow \bullet E + T$ 表示尚未读入任何 E 的右部符号；

$E \rightarrow E \bullet + T$ 表示已经读入 E ，还期待读入 $+T$ ；

$E \rightarrow E + T \bullet$ 表示已读入所有右部符号，可以归约。

移进-归约的过程

两个基本操作：[p.133]

- 移进 (Shift)：将一个输入记号压入栈顶
- 归约 (Reduce)：栈顶上的 n 个记号匹配某产生式右部时，将其弹出并压入该产生式左部

核心问题：如何确定移进和归约的时机？

LR 分析表

LR 分析表分为两部分：[p.134-137]

- ACTION 表：以状态为行，终结符（包括结束符 $\$$ ）为列，值为：
 - s_n ：移进当前记号，转移到状态 n
 - r_n ：用第 n 条产生式归约
 - accept：接受
 - 空白：出错
- GOTO 表：以状态为行，非终结符为列，值为转移到的状态 g_n

归约操作 (r_n) 的具体步骤：[p.135]

1. 从记号栈弹出产生式 n 的所有右部记号，同时从状态栈弹出相等数量的状态
2. 将产生式 n 的左部压入记号栈
3. 根据当前状态栈栈顶状态，查询 GOTO 表，转移状态

LR 分析器的四个动作：[p.137]

1. 移进 (shift)：ACTION[S_i, a] = S ，将 a 推进栈，新状态 $S_j = \text{GOTO}[S_i, a]$
2. 规约 (reduce)：ACTION[S_i, a] = r_d (规则 $A \rightarrow \beta$)，弹出 β 及相应状态，压入 A ，新状态 $S_j = \text{GOTO}[S_{i-|\beta|}, A]$
3. 接受 (accept)：成功结束
4. 出错 (error)：报告错误

5.1 LR(0) 分析算法

LR(0) 项目集规范族与 DFA 构造

LR(0) 分析表通过构造 DFA 来生成。DFA 的状态是项目集合（项集），DFA 的边是文法符号。
[p.145]

拓广文法（**Augmented Grammar**）：[p.152] 引入新的开始符号 S' 和产生式 $S' \rightarrow S\$$ （或 $S' \rightarrow E$ ），确保分析成功的唯一方式是使用 S' 产生式归约。

closure(I) 函数——求闭包：[p.154-156]

1. I 中的每个项目均加入 $\text{closure}(I)$
2. 如果 $A \rightarrow \alpha \bullet B\beta$ 在 $\text{closure}(I)$ 中，且 $B \rightarrow \gamma$ 是产生式，且 $B \rightarrow \bullet \gamma$ 不在 $\text{closure}(I)$ 中，则加入
3. 重复步骤 2 直到不再增加

goto(I,X) 函数：[p.157] $I' = \text{goto}(I, X) = \text{closure}(\{A \rightarrow \alpha X \bullet \beta \mid A \rightarrow \alpha \bullet X\beta \in I\})$

DFA 构造算法：[p.149-150]

初始项集 $I_0 = \text{closure}(\{[S' \rightarrow \cdot S\$]\})$

状态集合 $\text{states} = \{I_0\}$

while 存在未处理的状态 I :

 for each 文法符号 X :

$J = \text{goto}(I, X)$

 if J 非空且 J 不在 states 中:

states 添加 J

 添加边 $I \xrightarrow{X} J$

LR(0) 分析表生成规则

对每个 DFA 状态（项集）：[p.148, p.169, p.173]

- 若项集中包含 $A \rightarrow \alpha \bullet a\beta$ ，且有状态 j 通过终结符 a 可达，则 $\text{ACTION}[i][a] = s_j$ （移进）
- 若项集中包含 $A \rightarrow \alpha \bullet$ （点在末尾），则对所有终结符 $\text{ACTION}[i][a] = r_n$ （用产生式 n 归约）
- 若项集中包含 $S' \rightarrow S \bullet \$$ （开始产生式完成），则 $\text{ACTION}[i][\$] = \text{accept}$
- 若有通过非终结符 A 转移到状态 j 的边，则 $\text{GOTO}[i][A] = j$

LR(0) 的缺点

LR(0) 对于归约项直接对所有输入记号进行归约，导致两个问题：[p.179-184]

1. **延迟错误发现时机**：在不该归约时归约，导致错误在后续步骤才被发现
2. **移进-归约冲突**：某个状态同时有移进项和归约项，表中同一单元格出现多项

内在原因：LR(0) 项目只标识了“需要归约”，没有标识“何时归约”。因此对所有输入记号都进行归约。
[p.184]

5.2 SLR(1) 分析算法

SLR 的思想

与 LR(0) 基本相同，唯一区别在归约表项的生成规则：仅对属于 FOLLOW(A) 的输入记号添加对产生式 $A \rightarrow \alpha$ 的归约操作 r_n 。[p.185, p.189]

即：若某状态中包含项目 $A \rightarrow \alpha \bullet$ ，则仅对 $a \in \text{FOLLOW}(A)$ ， $\text{ACTION}[i][a] = r_n$ 。

SLR 的优点与局限

优点：

- 有可能减少需要归约的情况
- 有可能去除移进-归约冲突

缺点：仍然有冲突出现的可能。因为 FOLLOW 集判断“遇到产生式后面可能跟的所有输入都归约”，这个条件不够精确。[p.189, p.193]

典型例子： $S \rightarrow L = R \mid R, L \rightarrow *R \mid id, R \rightarrow L$ ，在状态 3 ($S \rightarrow L \bullet = R$ 和 $R \rightarrow L \bullet$ 共存) 中， $\text{FOLLOW}(R) = \{=, \$\}$ ，但对 $=$ 做 $R \rightarrow L$ 的归约是不正确的，因为此时的上下文是期望看到 $=$ 来匹配 $S \rightarrow L = R$ 。[p.190-193]

5.3 LR(1) 分析算法

LR(1) 项目的定义

LR(1) 项目定义为 $[X \rightarrow \alpha \bullet \beta, a]$ ，其中 a 是前看符号 (Lookahead)。含义：[p.194, p.196]

- 已读入 α
- 剩余的输入能够匹配 βa ；当 $\beta = \varepsilon$ 时，仅当下一输入符号为 a 时才能用该产生式归约

与 LR(0) 的关系：

- LR(0)：项目为 $[X \rightarrow \alpha \bullet \beta]$
- SLR：遇到 FOLLOW(X) 中的所有输入都归约
- LR(1)：在遇到推导过程中该产生式后面可能跟的输入才归约，更精确

[p.194]

前看符号的计算 (归纳法)

利用推导过程来传播前看符号：[p.195]

若推导链为： $S' \Rightarrow^* \alpha Y \beta \$ \Rightarrow \alpha \gamma \beta \$$ ，其中：

- $[S' \rightarrow S \bullet, a]$ 中 $a = \$$
- $[S \rightarrow \alpha Y \beta \bullet, b]$ 中 $b = \$$
- $[Y \rightarrow \gamma \bullet, c]$ 中 $c = \beta$ (若 $\beta \neq \varepsilon$)；或 $c = \$$ (若 $\beta = \varepsilon$)；或 $c = \text{FIRST_S}(\beta b)!$

LR(1) 的 closure 计算

对项目 $[X \rightarrow \alpha \bullet Y\beta, a]$, 添加 $[Y \rightarrow \bullet \gamma, b]$ 到项目集, 其中 $b \in \text{FIRST}_S(\beta a)$ 。[p.196]

LR(1) 的归约表项

对包含项目 $[X \rightarrow \alpha\beta\bullet, a]$ 的状态, 仅对输入 a 产生该产生式的归约项 r_n 。[p.196]

LR(1) 示例

对文法 $S' \rightarrow S\$$, $S \rightarrow L = R \mid R$, $L \rightarrow *R \mid id$, $R \rightarrow L$, LR(1) 成功解决了 SLR 的冲突问题。在状态 3 中: $[S \rightarrow L\bullet = R, \$]$ 对 $=$ 移进, $[R \rightarrow L\bullet, \$]$ 仅对 $\$$ 归约, 不再冲突。[p.197-199]

LALR 分析算法

LALR 的思想

LR(1) 的状态数量可能非常大。LALR (Look-Ahead LR) 合并具有类似项目集 (同心项目集) 的状态——仅前看符号不同的 LR(1) 项目集可以合并。[p.200-201]

同心 LR(1) 项目集: 核心 (第一分量集合) 相同, 仅搜索符不同的项目集。例如 $[B \rightarrow \bullet bB, \$]$ 与 $[B \rightarrow \bullet bB, b/a]$ 是核心相同的。[p.202]

LALR 的实现

1. 先构造 LR(1) 的各个项目集
2. 合并所有同心项目集, 前看符号取并集
3. 修改 ACTION 表和 GOTO 表以反映合并效果

[p.201-206]

二义性文法的 LR 处理

虽然二义性文法严格来说无法使用 LR 分析算法, 但实际应用中有些二义性文法更简洁。可以通过给 LR 分析器添加辅助规则来处理: [p.208-209]

- **优先级:** 定义操作符的优先级 (如 $*$ 优先于 $+$)
- **结合性:** 定义操作符是左结合还是右结合
- **悬空-else:** 规定 else 与最近的 if 匹配

示例: 对于二义性表达式文法 $E \rightarrow E + E \mid E * E \mid n$, 通过左结合和优先级 ($*$ $>$ $+$) 规则, 可以消除分析表中的冲突。[p.209]

5.4 LR 分析工具——YACC

YACC（Yet Another Compiler-Compiler）于 1975 年在 Unix 上实现，采用 LR(1) 分析算法。后续发展包括 GNU Bison 等。[p.211]

YACC 文件结构：[p.213]

声明部分：用户代码和YACC声明

%%

语法规则：上下文无关文法规则及语义动作

%%

用户代码：辅助函数

6 语法分析算法总结与比较

自顶向下 vs 自底向上

- 自顶向下分析（LL 类）：适用手工编码，文法受限（需要消除左递归和提取左公因子），如递归下降、LL(1)
- 自底向上分析（LR 类）：适用自动生成工具，对文法的适用范围更广，如 LR(0)、SLR(1)、LR(1)、LALR(1)

[p.127, p.219]

文法包含关系

正则文法 $\subset$ 无回溯文法 (LL(1)) $\subset$ LR(0) $\subset$ SLR(1) $\subset$ LALR(1) $\subset$ LR(1) $\subset$ CFG

[p.127]

LL(1) 和 LR(1) 有交集但不互相包含。[p.127]

各类 LR 分析算法的比较

算法	前看符号数	归约条件	表达能力	[p.189, p.194, p.201]
LR(0)	0	无限制地对所有输入归约	最弱	
SLR(1)	1(FORLLOW)	FOLLOW 集中的输入才归约	中等	
LR(1)	1(精确)	推导上下文中精确的前看符号	最强 (状态多)	
LALR(1)	1(合并)	合并同心 LR(1) 项目集后	接近 LR(1)(状态少)	

7 核心考点速记

关键公式与定义

1. 文法四元组： $G = (T, N, P, S)$ 或 $G = (V_N, V_T, S, P)$, $V_N \cap V_T = \emptyset$ [p.15, p.26]
2. 推导： $S \overset{*}{\Rightarrow} u$ （句型）; $S \overset{*}{\Rightarrow} u \in V_T^*$ （句子） [p.42]

3. 语言: $L(G) = \{u \mid S \xRightarrow{*} u, u \in V_T^*\}$ [p.42]
4. 二义性: 存在句子 s 有两棵不同的语法树 [p.50]
5. **NULLABLE**: $X \rightarrow \varepsilon$ 或 $X \rightarrow Y_1 \dots Y_n$ 且所有 Y_i **NULLABLE** [p.97]
6. **FIRST**(X): $\{a \in V_T \mid X \xRightarrow{*} a \dots\} \cup \{\varepsilon \text{ 若 } X \xRightarrow{*} \varepsilon\}$ [p.92, p.100]
7. **FOLLOW**(X): $\{a \in V_T \mid S \xRightarrow{*} \dots Xa \dots\}$ [p.104]
8. **FIRST** $_S(\beta_1 \dots \beta_n)$: 从 $\beta_1 \dots \beta_n$ 推导出的开头终结符集合 [p.90, p.107]

FIRST 集计算步骤

1. 计算 **NULLABLE** 集合 (不动点迭代)
2. 初始化所有 **FIRST**(X) = $\emptyset$
3. 对每条产生式 $X \rightarrow Y_1 Y_2 \dots Y_n$:
 - 对 $a \in \text{FIRST}(Y_1)$: $\text{FIRST}(X) \cup = \{a\}$
 - 若 Y_1 **NULLABLE**, 对 $a \in \text{FIRST}(Y_2)$: $\text{FIRST}(X) \cup = \{a\}$
 - 继续判断 Y_2 是否 **NULLABLE**, 以此类推
 - 若所有 Y_i 都 **NULLABLE**: $\text{FIRST}(X) \cup = \{\varepsilon\}$
4. 重复步骤 3 直到所有 **FIRST** 集不再变化

[p.100-102]

FOLLOW 集计算步骤

1. $\text{FOLLOW}(S) \cup = \{\$ \}$ (S 是开始符号)
2. 对每条产生式, 从右向左扫描:
 - 维护一个 temp 集合 = **FOLLOW**(左部)
 - 对于右部最后一个符号 M : $\text{FOLLOW}(M) \cup = \text{temp}$
 - 若 M **NULLABLE**, $\text{temp} \cup = \text{FIRST}(M)$, 否则 $\text{temp} = \text{FIRST}(M)$
 - 向前继续处理
3. 重复直到不动点

[p.105]

LL(1) 分析表构造步骤

1. 消除左递归、提取左公因子
2. 计算 NULLABLE 集
3. 计算 FIRST 集和 FOLLOW 集（不动点算法）
4. 计算每条产生式的 FIRST_S 集
5. 填充分析表：
 - 对每条产生式 $A \rightarrow \beta$ ，对 $\text{FIRST_S}(\beta)$ 中每个 a : $\text{table}[A][a] = \text{产生式编号}$
 - 若 $\beta \Rightarrow^* \varepsilon$ ，对 $\text{FOLLOW}(A)$ 中每个 b : $\text{table}[A][b] = \text{产生式编号}$

[p.91-109]

消除左递归的通用公式

对 $P \rightarrow P\alpha_1 \mid \dots \mid P\alpha_m \mid \beta_1 \mid \dots \mid \beta_n$:

$P \rightarrow \backslash\text{beta}_1 P' \mid \dots \mid \backslash\text{beta}_n P'$

$P' \rightarrow \backslash\text{alpha}_1 P' \mid \dots \mid \backslash\text{alpha}_m P' \mid \backslash\text{varepsilon}$

[p.73]

LR(0) DFA 构造算法

function closure(I):

 重复直到不再变化:

 for each $A \rightarrow \backslash\text{alpha}.B\backslash\text{beta}$ in I:

 for each $B \rightarrow \backslash\text{gamma}$:

 if $B \rightarrow \backslash\text{gamma}$ not in I:

 add $B \rightarrow \backslash\text{gamma}$ to I

 return I

function goto(I, X):

 J = {}

 for each $A \rightarrow \backslash\text{alpha}.X\backslash\text{beta}$ in I:

 add $A \rightarrow \backslash\text{alpha} X.\backslash\text{beta}$ to J

 return closure(J)

$I_0 = \text{closure}(\{S' \rightarrow \backslash S \$\})$

states = { I_0 }

for each I in states:

 for each X (文法符号):

 J = goto(I, X)

```
if J not in states: add J
    建立从I到J的边(标注X)
```

[p.149-150, p.154-157]

LR 分析表生成规则对比

规则	LR(0)	SLR(1)	LR(1)
移进 $A \rightarrow \alpha \bullet a\beta$	s_j	s_j	s_j
转移 A (GOTO)	g_j	g_j	g_j
归约 $A \rightarrow \alpha \bullet$	对所有 $a: r_n$	对 $a \in \text{FOLLOW}(A): r_n$	对 a (前看符号): r_n
接受	仅 \$	仅 \$	仅 \$

[p.148, p.185,

p.196]

LR(1) closure 计算

对项目 $[X \rightarrow \alpha \bullet Y\beta, a]$:

- 对每个产生式 $Y \rightarrow \gamma$
- 添加 $[Y \rightarrow \bullet\gamma, b]$ 到项目集
- 其中 $b \in \text{FIRST}_S(\beta a)$ (如果 βa 全为 NULLABLE 则为 a)

[p.196-197]

LL(1) 文法判定条件速查

两条产生式 $A \rightarrow \alpha \mid \beta$, 以下条件必须全部满足: [p.113]

1. α, β 都不能 $\Rightarrow^* \varepsilon$: $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$
2. α 能 $\Rightarrow^* \varepsilon$: $\text{FIRST}(\beta) \cap \text{FOLLOW}(A) = \emptyset$
3. β 能 $\Rightarrow^* \varepsilon$: $\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset$

LR/SLR/LR(1)/LALR 的冲突判定

- 移进-归约冲突: 某状态同时有移进项和归约项, 且对同一输入分别需要移进和归约
- 归约-归约冲突: 某状态有两个不同产生式的归约项 (且对同一输入都可以归约)
- SLR 可以解决 LR(0) 的部分冲突, 但不能解决所有
- LR(1) 可以解决 SLR 的冲突 (通过精确前看符号)
- LALR(1) 合并同心项目后, 可能会引入新的归约-归约冲突 (但实际语言中很少见)

[p.189-206]

期末考试常见题型

1. 给定文法，判断类型（0/1/2/3 型），求 $L(G)$
2. 给定文法和句子，写出最左/最右推导，画语法树
3. 判断文法是否有二义性
4. 消除左递归、提取左公因子
5. 计算 FIRST 集、FOLLOW 集、NULLABLE 集
6. 判定是否为 LL(1) 文法，构造 LL(1) 预测分析表
7. 构造 LR(0) 项目集规范族及 DFA
8. 构造 SLR(1) 分析表
9. 构造 LR(1) 项目集和分析表
10. 利用 LR 分析表分析给定输入串

[p.220-223]